Raft Consensus: Lecture Notes

Leader election, log replication and safety

1. The problem Raft solves

Many distributed systems keep critical state, such as configuration or metadata, on several servers so that the state survives machine failures. A common way to build such a system is a replicated state machine: every server stores a log of commands and applies the commands in log order to a deterministic state machine. If every server applies the same commands in the same order, every server ends up in the same state. A consensus algorithm keeps the replicated logs consistent.

Raft is a consensus algorithm designed by Diego Ongaro and John Ousterhout and presented at the USENIX Annual Technical Conference in 2014. Its stated design goal was understandability. Raft provides the same fault tolerance as Multi-Paxos, but it separates the problem into relatively independent parts: leader election, log replication and safety.

Majorities and fault tolerance

Raft makes progress only when a majority of the servers in the cluster, called a quorum, can communicate. Any two majorities of the same cluster share at least one server, and this overlap is what prevents two conflicting decisions from both being accepted. A cluster of 2f + 1 servers can tolerate f failed servers. A three-server cluster therefore tolerates one failure, and a five-server cluster tolerates two failures. If more servers fail, the cluster stops accepting new writes until enough servers recover, but it never returns inconsistent results.

Adding a server to make an even-sized cluster does not improve fault tolerance. A four-server cluster needs three servers for a majority, so it still tolerates only one failure, the same as a three-server cluster.

Roles

At any time each server is in one of three states. A follower is passive: it responds to requests from leaders and candidates. A candidate is a server that is trying to become leader. A leader handles all client requests that modify state and replicates log entries to the followers. In normal operation there is exactly one leader and all other servers are followers.

2. Leader election

Raft divides time into terms, numbered with consecutive integers. Each term begins with an election. If a candidate wins, it serves as leader for the rest of the term. If an election ends without a winner, the term ends with no leader and a new term begins. Terms act as a logical clock: every message carries the sender's current term, a server that sees a higher term immediately updates its own term and becomes a follower, and requests with an outdated term are rejected.

The leader maintains its authority by sending periodic heartbeats to all followers. In Raft, a heartbeat is simply an AppendEntries RPC that carries no log entries. A follower that receives no communication from a valid leader for a period called the election timeout assumes that the leader has failed and starts an election.

To start an election, a follower increments its current term, transitions to the candidate state, votes for itself and sends RequestVote RPCs to all other servers. Each server votes for at most one candidate in a given term, on a first-come, first-served basis, subject to the safety restriction described in section 4. The candidate becomes leader if it receives votes from a majority of the servers in the full cluster. It then sends heartbeats to establish its authority and prevent new elections.

Split votes and randomized timeouts

If several followers become candidates at the same time, the votes may be split so that no candidate obtains a majority. Each candidate then times out and starts a new election with a higher term. Without further measures, split votes could repeat indefinitely.

Raft makes split votes rare by choosing each server's election timeout randomly from a fixed interval. The Raft paper suggests an interval such as 150 to 300 milliseconds. Because the timeouts differ, in most cases a single server times out first, wins the election and sends heartbeats before any other server times out. The election timeout must be much larger than the time needed to send a message to the other servers, and much smaller than the average time between server failures.

3. Log replication

Once a leader has been elected, it begins serving client requests. Each request contains a command for the replicated state machine. The leader appends the command to its log as a new entry and then sends AppendEntries RPCs in parallel to the other servers to replicate the entry. Each log entry stores the command, the term in which the leader received it, and its index, an integer that identifies its position in the log.

A log entry is committed once the leader that created it has replicated it on a majority of the servers. Committed entries are durable and will eventually be executed by every available state machine. When an entry is committed, all preceding entries in the leader's log are committed as well. The leader keeps track of the highest index it knows to be committed and includes that index in future AppendEntries RPCs, so followers learn which entries they may apply to their own state machines.

The consistency check

When sending an AppendEntries RPC, the leader includes the index and term of the entry in its log that immediately precedes the new entries. These fields are called prevLogIndex and prevLogTerm. A follower refuses the new entries if it does not have an entry at prevLogIndex whose term matches prevLogTerm. This consistency check maintains the Log Matching Property: if two logs contain an entry with the same index and term, then the logs are identical in all entries up to and including that index.

Repairing inconsistent followers

Crashes can leave a follower's log different from the leader's: the follower may be missing entries, or it may contain extra uncommitted entries from an earlier term. Raft handles this by forcing followers' logs to duplicate the leader's log. When the consistency check fails, the leader decrements its record of the follower's next index and retries, until it finds the point where the two logs agree. The follower then deletes any conflicting entries after that point and appends the leader's entries. A leader never overwrites or deletes entries in its own log.

4. Safety

Leader election and log replication alone are not enough to guarantee that every state machine executes the same commands in the same order. A follower that missed several committed entries could be elected leader and overwrite them. Raft prevents this with a restriction on which servers may be elected.

The election restriction

A candidate must contact a majority of the cluster to be elected, and every committed entry is present on at least one server in any majority. The RequestVote RPC therefore includes information about the candidate's log, and a voter denies its vote if its own log is more up-to-date than the candidate's log. Raft decides which of two logs is more up-to-date by comparing the index and term of their last entries. If the last entries have different terms, the log whose last entry has the later term is more up-to-date. If the logs end with the same term, the longer log is more up-to-date.

This restriction guarantees the Leader Completeness Property: if a log entry is committed in a given term, that entry will be present in the logs of the leaders for all higher-numbered terms.

Committing entries from previous terms

A subtle case arises when a new leader finds entries from earlier terms that are stored on a majority of servers but were never known to be committed. Counting replicas is not sufficient to commit such entries, because a future leader could still overwrite them. Raft therefore never commits log entries from previous terms by counting replicas. Only entries from the leader's current term are committed by counting replicas; once such an entry is committed, all earlier entries are committed indirectly through the Log Matching Property.

State Machine Safety

Together, these rules give the State Machine Safety Property: if a server has applied a log entry at a given index to its state machine, no other server will ever apply a different log entry for the same index.

5. Practical concerns

Cluster membership changes

Replacing servers or changing the size of the cluster must not create a moment in which two different majorities could elect two leaders for the same term. The original Raft paper uses joint consensus, a transitional configuration in which agreement requires separate majorities from both the old and the new configurations. Ongaro's dissertation also describes a simpler approach that adds or removes one server at a time, because any majority of the old configuration then overlaps with any majority of the new one.

Log compaction

A log cannot grow forever. Raft servers take snapshots of their state machine and discard the log entries that the snapshot covers. A leader uses the InstallSnapshot RPC to send its snapshot to a follower that has fallen so far behind that the entries it needs have already been discarded.

Reads and performance

A naive leader that answers read requests from its local state could return stale data if it has been replaced by a newer leader without knowing it. Linearizable reads therefore require extra care, for example confirming leadership with a round of heartbeats before answering, or using leader leases that rely on bounded clock drift.

In the common case, a write is committed after a single round of AppendEntries RPCs from the leader to a majority of the cluster.

Scalability

Raft is not a technique for scaling write throughput. Every write is sent through the single leader and must be replicated to a majority, so adding servers improves fault tolerance and read capacity but does not increase write throughput; larger clusters can even make writes slower, because the leader must replicate each entry to more followers. For this reason, production clusters usually contain three or five servers. Systems that need to scale beyond one leader partition their data into many independent Raft groups, each with its own leader, as CockroachDB and TiKV do for ranges of their key space.

Raft in practice

Raft is used for coordination and metadata in widely deployed systems, including etcd, which stores the cluster state of Kubernetes, and HashiCorp Consul.